北京邮电大学

《网络安全》课程实验报告

实验名称：实验 4：防火墙课堂实验

学生姓名：

学　　号：

班　　级：

实验 4：防火墙课堂实验

一、实验目的

1.学习 iptables 防火墙基本操作。

2.设置 iptables 防火墙的包过滤规则，分别实现以下功能：

（1）禁止所有主机 ping 本地主机；

（2）仅允许某特定 IP 主机 ping 本地主机；

（3）允许每 10 秒钟通过 1 个 ping 包；

（4）阻断来自某个 mac 地址的数据包。

3.设置 iptables 规则，实现特定远端主机 SSH 连接本地主机

二、实验环境

1.配置防火墙的虚拟机：kali

2.验证防火墙功能的虚拟机：ubuntu

3.软件：iptables

4.Windows 平台 SSH 工具：putty

（网络连接均设置为仅主机模式）

三、实验过程与结果

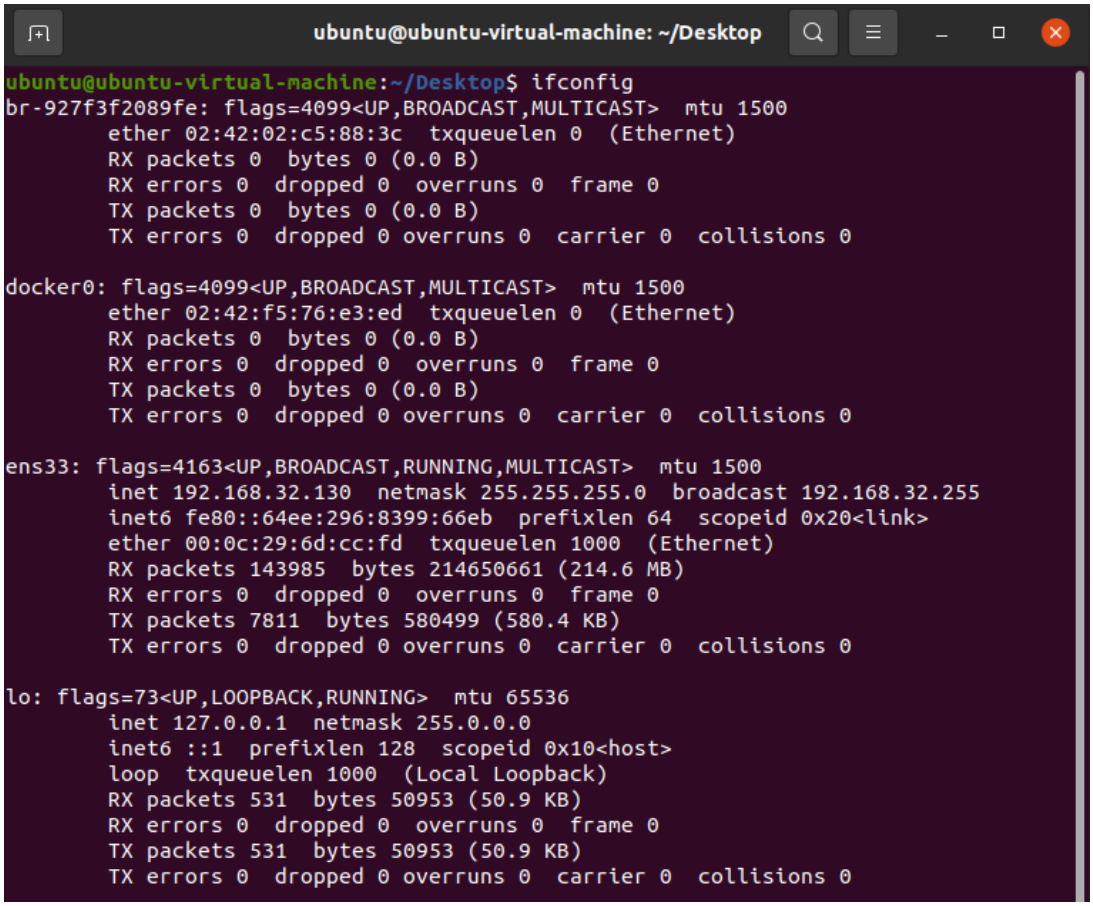
3.1 步骤一：查看主机信息并实现 ping 通

1.kali 终端输入“ifconfig”，可知 IP 地址为 192.168.32.129

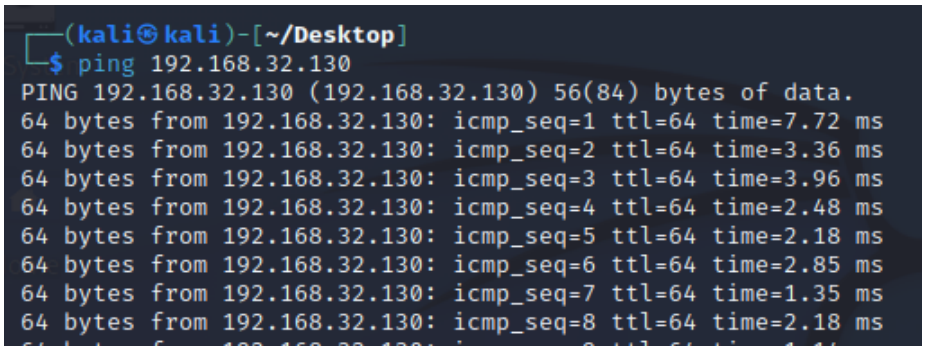
```
┌──(kali㉿kali)-[~/Desktop]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.32.129  netmask 255.255.255.0  broadcast 192.168.32.255
        inet6 fe80::4dac:cf91:1287:8621  prefixlen 64  scopeid 0x20<link>
        ether 00:0c:29:a5:82:6c  txqueuelen 1000  (Ethernet)
        RX packets 7638  bytes 532179 (519.7 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 531725  bytes 31988860 (30.5 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 543  bytes 46000 (44.9 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 543  bytes 46000 (44.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

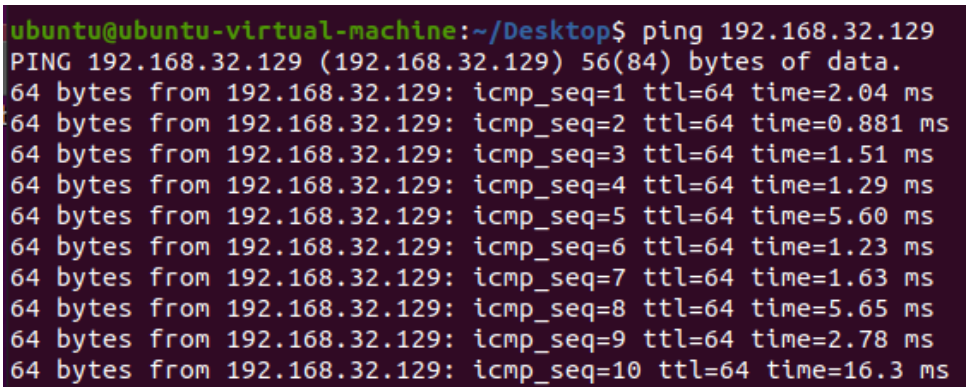
2.ubuntu 终端输入“ifconfig”，可知 IP 地址为 192.168.32.130



3.kali ping 通 ubuntu。



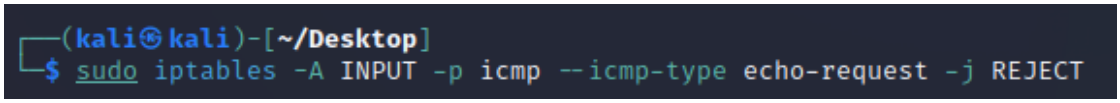
4.ubuntu ping 通 kali。



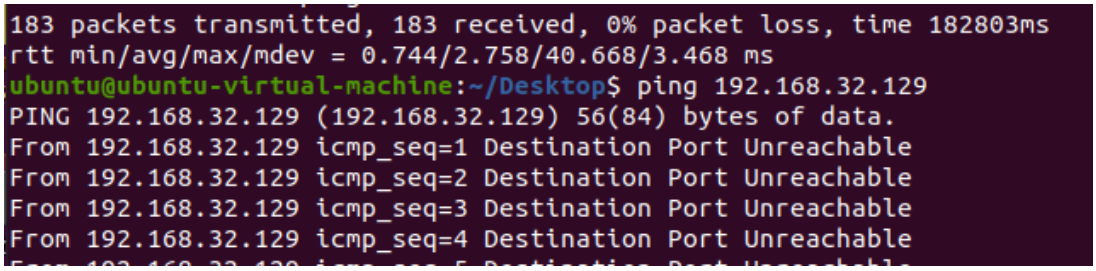
3.2 步骤二：设置 iptables 防火墙的包过滤规则

1.在 kali 终端输入以下命令，禁止所有主机 ping 本地主机。

```
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j REJECT
```

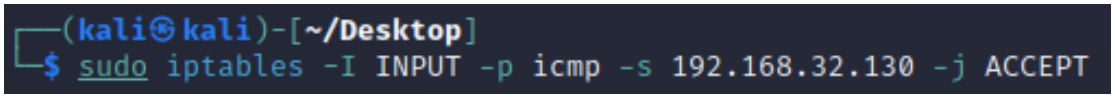


2.此时再次在 ubuntu 中尝试 ping kali 发现无法 ping 通。

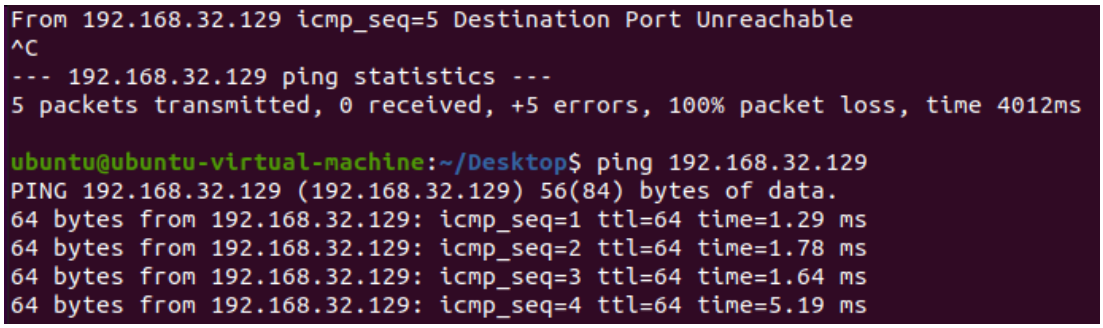


3.在 kali 终端输入以下命令，仅允许指定 ip 地址的 ubuntu 主机 ping 主机 kali。

```
sudo iptables -I INPUT -p icmp -s 192.168.32.130 -j ACCEPT
```

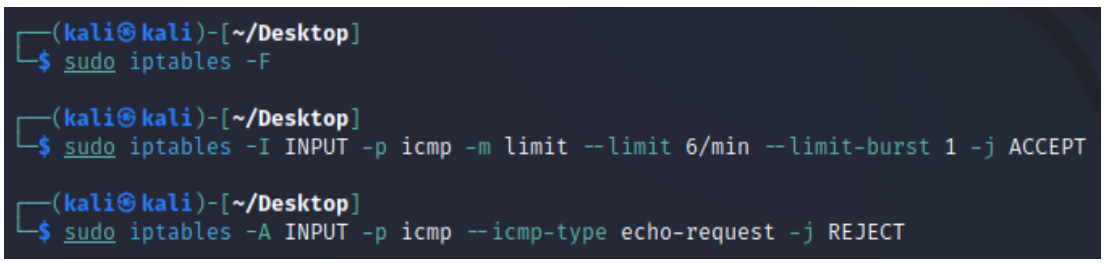


4.此时 ubuntu 就又能 ping 通 kali 了。



5.为了保证两条规则的先后顺序，首先输入 **sudo iptables -F** 清空全部规则，再输入 **sudo iptables -I INPUT -p icmp -m limit --limjit 6/min --limit-burst 1 -j ACCEPT** 命令加入允许每十秒通过一个 ping 包的规则，最后再加入禁止任何主机 ping 本地主机的规则，保证每十秒一个 ping 包的规则优先于后者。

```
sudo iptables -F
sudo iptables -I INPUT -p icmp -m limit --limit 6/min --limit-burst 1 -j ACCEPT
sudo iptables -A INPUT -p icmp --icmp-type echo-request -j REJECT
```



6.此时利用 **sudo iptables -L** 查看规则列表如下图所示，ACCEPT 优先于 REJECT。

sudo iptables -L

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     icmp --  anywhere             anywhere             limit: avg 6/min burst 1
REJECT     icmp --  anywhere             anywhere             icmp echo-request reject-with icm
p-port-unreachable

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

7.设置完成后，此时 ubuntu ping kali，每十秒成功发送一个 ping 包，其余时间无法 ping 通。

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ ping 192.168.32.129
PING 192.168.32.129 (192.168.32.129) 56(84) bytes of data.
64 bytes from 192.168.32.129: icmp_seq=1 ttl=64 time=1.71 ms
From 192.168.32.129 icmp_seq=2 Destination Port Unreachable
From 192.168.32.129 icmp_seq=3 Destination Port Unreachable
From 192.168.32.129 icmp_seq=4 Destination Port Unreachable
From 192.168.32.129 icmp_seq=5 Destination Port Unreachable
From 192.168.32.129 icmp_seq=6 Destination Port Unreachable
From 192.168.32.129 icmp_seq=7 Destination Port Unreachable
From 192.168.32.129 icmp_seq=8 Destination Port Unreachable
From 192.168.32.129 icmp_seq=9 Destination Port Unreachable
From 192.168.32.129 icmp_seq=10 Destination Port Unreachable
64 bytes from 192.168.32.129: icmp_seq=11 ttl=64 time=2.23 ms
From 192.168.32.129 icmp_seq=12 Destination Port Unreachable
From 192.168.32.129 icmp_seq=13 Destination Port Unreachable
From 192.168.32.129 icmp_seq=14 Destination Port Unreachable
From 192.168.32.129 icmp_seq=15 Destination Port Unreachable
From 192.168.32.129 icmp_seq=16 Destination Port Unreachable
From 192.168.32.129 icmp_seq=17 Destination Port Unreachable
From 192.168.32.129 icmp_seq=18 Destination Port Unreachable
```

8.再次清空 iptables 规则后，在 kali 终端输入 **sudo iptables -I INPUT -p icmp -m mac –mac-source 00:0c:29:71:1d:82 -j DROP**，阻断来自 ubuntu 的 Mac 地址 00:0c:29:71:1d:82 的数据包。

sudo iptables -F sudo iptables -I INPUT -p icmp -m mac --mac-source 00:0c:29:6d:cc:fd -j DROP

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -F

┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -I INPUT -p icmp -m mac --mac-source 00:0c:29:6d:cc:fd -j DROP
```

9.查看此时的规则列表如下，只有一条 DROP 规则。

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       icmp --  anywhere             anywhere             MAC 00:0c:29:6d:cc:fd

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

10.此时 ubuntu 无法 ping 通 kali 了。

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ ping 192.168.32.129
PING 192.168.32.129 (192.168.32.129) 56(84) bytes of data.
^C
--- 192.168.32.129 ping statistics ---
12 packets transmitted, 0 received, 100% packet loss, time 11245ms
```

3.3 步骤三：实现特定远端主机 SSH 连接本地主机

1.在 kali 终端分别执行 **sudo /etc/init.d/ssh start** 和 **sudo /etc/init.d/ssh status** 命令，启动 kali 的 SSH 服务并查看 ssh 服务器状态，使得客户端可以通过 22 端口连接。

sudo /etc/init.d/ssh start sudo /etc/init.d/ssh status

```
┌──(kali㉿kali)-[~]
└─$ sudo /etc/init.d/ssh start
Starting ssh (via systemctl): ssh.service.

┌──(kali㉿kali)-[~]
└─$ sudo /etc/init.d/ssh status
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; disabled; vendor preset: disabled)
     Active: active (running) since Thu 2025-11-13 13:26:26 CST; 29min ago
       Docs: man:sshd(8)
             man:sshd_config(5)
    Process: 1575 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 1576 (sshd)
      Tasks: 1 (limit: 2283)
     Memory: 2.0M
        CPU: 133ms
     CGroup: /system.slice/ssh.service
             └─1576 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Nov 13 13:26:26 kali systemd[1]: Starting OpenBSD Secure Shell server...
Nov 13 13:26:26 kali sshd[1576]: Server listening on 0.0.0.0 port 22.
Nov 13 13:26:26 kali sshd[1576]: Server listening on :: port 22.
Nov 13 13:26:26 kali systemd[1]: Started OpenBSD Secure Shell server.
```

2.在 ubuntu 终端输入 **ssh kali@192.168.32.129**，ubuntu 客户端 SSH 连接服务器 kali。

```
ubuntu@ubuntu-virtual-machine:~/Desktop$ ssh kali@192.168.32.129
The authenticity of host '192.168.32.129 (192.168.32.129)' can't be established.
ECDSA key fingerprint is SHA256:B2Ijer6M5knyqvbwfDltAGJlD/3R9yx7/RluVnt5+Uc.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.32.129' (ECDSA) to the list of known hosts.
kali@192.168.32.129's password:
Linux kali 5.18.0-kali5-amd64 #1 SMP PREEMPT_DYNAMIC Debian 5.18.5-1kali6 (2022-07-
07) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
┌──(kali㉿kali)-[~]
└─$
```

3.在 windows 11 主机命令行中也输入 **ssh kali@192.168.32.129**，主机客户端 SSH 连接服务器 kali。

```
kali@kali: ~
Windows PowerShell
版权所有 (C)  Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

加载个人及系统配置文件用了 749 毫秒。
(base) PS C:\Users\1\Desktop> ssh kali@192.168.32.129
kali@192.168.32.129's password:
Linux kali 5.18.0-kali5-amd64 #1 SMP PREEMPT_DYNAMIC Debian 5.18.5-1kali6 (2022-07-07) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Nov 13 14:35:34 2025 from 192.168.32.130
┌──(kali㉿kali)-[~]
└─$
```

4.在 kali 终端再次查看 ssh 服务器状态，发现 SSH 服务器上留下记录，两个客户端成功连接服务器。

```
sudo journalctl -u ssh -f
```

```
┌──(kali㉿kali)-[~]
└─$ sudo journalctl -u ssh -f
```

```
Nov 13 14:35:34 kali sshd[14900]: Accepted password for kali from 192.168.32.130 port 45132 ssh2
Nov 13 14:35:34 kali sshd[14900]: pam_unix(sshd:session): session opened for user kali(uid=1000) b
y (uid=0)
Nov 13 14:36:13 kali sshd[15085]: Accepted password for kali from 192.168.32.1 port 56757 ssh2
Nov 13 14:36:13 kali sshd[15085]: pam_unix(sshd:session): session opened for user kali(uid=1000) b
y (uid=0)
```

5.再次清空 iptables 规则后，在 kali 终端输入 **sudo iptables -I INPUT -p tcp --dport 22 -s 192.168.32.130 -j ACCEPT** 允许远端主机 ubuntu 进行 SSH 连接，再输入 **sudo iptables -A INPUT -p tcp --dport 22 -j DROP** 禁止其他远端主机 SSH 连接。

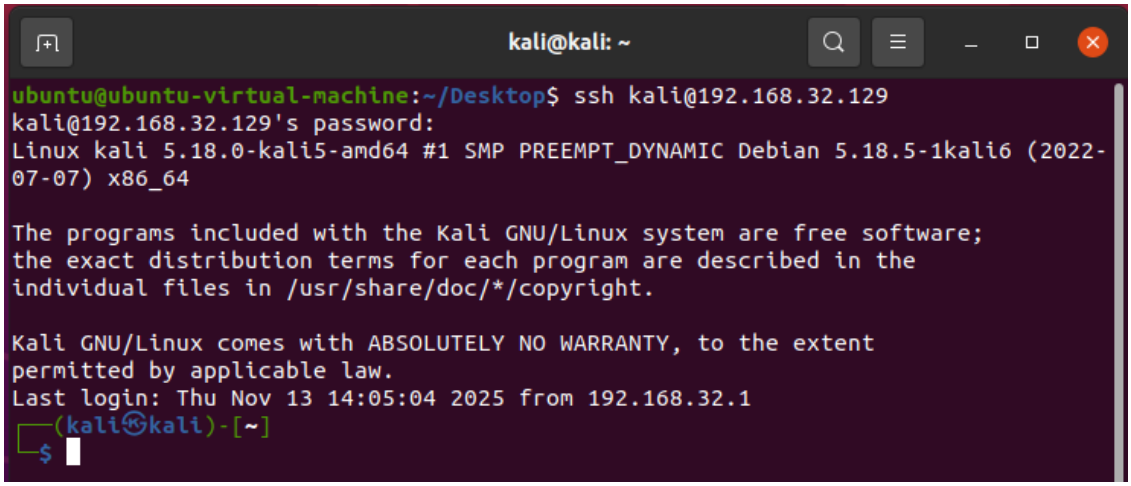
```
sudo iptables -F
sudo iptables -I INPUT -p tcp --dport 22 -s 192.168.32.130 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -F

┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -I INPUT -p tcp --dport 22 -s 192.168.32.130 -j ACCEPT

┌──(kali㉿kali)-[~/Desktop]
└─$ sudo iptables -A INPUT -p tcp --dport 22 -j DROP
```

6.在 iptables 中允许连接的客户端 1ubuntu 仍可远程 SSH 连接。



7.客户端 2 本机 windows 11 已经无法连接，显示“client_loop: send disconnect: Connection reset”。

```
Windows PowerShell
版权所有 (C)  Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

加载个人及系统配置文件用了 1068 毫秒。
(base) PS C:\Users\1> ssh kali@192.168.32.129
ssh: connect to host 192.168.32.129 port 22: Connection timed out
(base) PS C:\Users\1>
```

四、实验总结

在本次关于 iptables 防火墙配置的实验中，我深入学习了 iptables 这一 Linux 核心安全工具的基本操作与原理知识。iptables 作为 Linux 系统中用于配置 IPv4 数据包过滤规则、实现网络地址转换（NAT）以及管理数据包队列的强大工具，其价值在于允许用户通过定义精细的规则集，主动控制网络数据包的流向与处理方式，从而构建起灵活而有效的网络安全策略，实现包括端口控制、访问限制及防御网络攻击在内的多种功能。

实验过程中，我首先熟悉了 iptables 的基本命令结构，包括对 INPUT、FORWARD、OUTPUT 等链的操作。随后，我着手设置具体的包过滤规则以完成一系列预定目标。首要任务是掌握如何限制 ICMP 协议，我们通过添加规则，将所有传入的 ICMP 回显请求包（即 ping 请求）丢弃，从而实现了禁止所有主机 ping 通本地主机的效果，这让我直观地理解了防火墙如何基于协议类型进行访问控制。

在此基础上，实验进一步提升了规则的精确性。我学习了如何使用-s 参数指定源 IP 地址，通过设置一条允许特定 IP 主机 ping 的规则，并确保其位于默认的禁止规则之前，成功实现了仅允许该授权主机进行 ping 操作，而其他任何主机的请求均被阻断。这一正一反的规则设置，深刻体现了 iptables 规则顺序的重要性以及“白名单”策略的实现方法。

为了应对更复杂的流量整形需求，我们尝试使用了 limit 模块。通过配置-m limit --limit 6/minute（即每分钟 6 个，约合每 10 秒 1 个）这样的参数，我们实现了对 ICMP 请求的频率限制。当 ping 包的到来速率超过此阈值时，超出的请求会被自动丢弃。这个实验环节让我认识到，iptables 不仅能够进行简单的“允许”或“拒绝”判断，还能对流量速率进行智能管理，这对于缓解网络洪泛攻击等场景具有实用意义。

此外，我们还探索了基于数据链路层信息的过滤方式，即根据源 MAC 地址来阻断数据包。这通过-m mac --mac-source 参数实现，尽管 MAC 地址过滤在跨路由器的场景下作用有限，但它让我了解到防火墙规则可以作用于网络模型的不同层次，为局域网内的安全控制提供了另一种手段。

最后，实验的重点转向了应用层的访问控制，即管理 SSH 远程连接。我们通过添加规则，允许来自特定远端 IP 地址的 TCP 22 端口连接，同时可以配置默认策略拒绝其他所有 SSH 连接尝试。这个过程巩固了我对基于端口和状态（如新建连接）进行过滤的理解，这是保障服务器远程管理安全的基础。

总而言之，本次实验不仅让我熟悉了 iptables 的各种命令参数和规则链概念，更重要的是通过亲手实践，使我清晰地认识到如何将理论上的安全策略转化为具体、可执行的防火墙规则。iptables 的高度可定制性确实使其成为 Linux 系统网络管理与安全防御中不可或缺的利器。通过这次实践，我初步具备了利用 iptables 构建基础主机防火墙的能力，并对网络数据包的流动与控制有了更为具象化的认识。